TPUs, NPUs, and Domain-Specific Accelerators: When Specialization Beats SIMT

Aksel Aghajanyan
Technical Report — Knowledge Base Archive

Abstract

Central Processing Units (CPUs) and Graphics Processing Units (GPUs) occupy the general-purpose and data-parallel poles of modern heterogeneous computing, yet production systems increasingly rely on **domain-specific accelerators**—Tensor Processing Units (TPUs), Neural Processing Units (NPUs), Field-Programmable Gate Arrays (FPGAs), and fixed-function Application-Specific Integrated Circuits (ASICs)—to extract performance that programmable SIMT hardware cannot match at equal power or silicon cost. This report develops a principled framework for when specialization wins: algorithm stability, arithmetic structure, memory access regularity, deployment volume, and latency constraints. We examine systolic-array TPUs, edge NPUs, reconfigurable FPGA pipelines, and media ASICs (including HEVC hardware encoders), and compare each against GPU SIMT execution using the roofline model, energy–performance tradeoffs, and case studies in neural inference and video compression.

I. Introduction

The prior analysis of CPU versus GPU architectures established that processor choice is governed by **workload fit**: latency-optimized MIMD cores for irregular control flow, throughput-optimized SIMT cores for large-scale data parallelism. That dichotomy, however, is incomplete. Modern datacenters, mobile SoCs, and edge devices embed a **spectrum of accelerators** alongside the CPU and GPU, each sacrificing programmability to maximize efficiency on a narrow class of computations.

The phrase "domain-specific accelerator" (DSA) denotes hardware whose instruction set, datapath, and memory hierarchy are co-designed for a fixed algorithmic pattern—matrix multiply–accumulate in a TPU, convolution and depthwise ops in an NPU, motion search and transform coding in a video encoder ASIC. Unlike a GPU, which executes arbitrary CUDA kernels subject to SIMT constraints, a DSA typically exposes a **compiler target** or fixed API (XLA for TPUs, CoreML/NNAPI for NPUs, NVENC for media blocks) and hardwires the inner loop.

Confusion arises when practitioners treat all accelerators as "faster GPUs." They are not. A TPU v4 pod excels at large-batch matrix operations with predictable dataflow but cannot run arbitrary branch-heavy graph algorithms. An FPGA can implement a custom HEVC motion-search pipeline at wire speed but requires months of RTL or HLS engineering per design. A smartphone NPU delivers milliwatt-class inference but supports only a subset of ONNX operators. The engineering question is not "GPU or accelerator" but **where on the specialization spectrum** a workload belongs.

This report compares DSAs across four axes: the specialization spectrum and design economics, TPU and systolic-array architecture, NPUs and edge inference, FPGAs and reconfigurable pipelines, fixed-function media ASICs (with emphasis on HEVC), and decision criteria for when specialization beats SIMT.

—

II. The Specialization Spectrum

A. From General Purpose to Fixed Function

Compute hardware may be ordered by decreasing flexibility and increasing peak efficiency on a target workload:

1) **CPU** — fully programmable, MIMD, optimized for latency and irregular parallelism.
2) **GPU** — programmable SIMT, optimized for throughput on data-parallel kernels.
3) **Structured accelerators (TPU, NPU)** — programmable within a **operator graph** or tensor algebra; datapath hardwired for GEMM, convolutions, and reductions.
4) **Reconfigurable logic (FPGA)** — gate-level reprogrammability; efficiency between ASIC and GPU for stable pipelines.

5) **Fixed-function ASIC** — hardwired state machines and arithmetic units for one algorithm (video encode/decode, crypto, DSP).

Let E denote energy per operation, P peak throughput on the target kernel, and F flexibility (ability to run new algorithms without silicon respin). Empirically:

$$F_{\mathrm{CPU}} > F_{\mathrm{GPU}} > F_{\mathrm{TPU}} > F_{\mathrm{FPGA}} > F_{\mathrm{ASIC}}, \quad E_{\mathrm{ASIC}} \lesssim E_{\mathrm{TPU}} < E_{\mathrm{GPU}} \ll E_{\mathrm{CPU}} \text{ (on-target)} \tag{1}$$

Specialization trades F for lower E and higher P on the chosen kernel family.

B. The Economics of Specialization

Fixed-function ASICs amortize non-recurring engineering (NRE) cost over **unit volume**. If NRE is C_{NRE} and per-unit marginal cost is c_{ASIC}, break-even against a programmable GPU with per-unit cost c_{GPU} occurs at volume V^* satisfying:

$$C_{\mathrm{NRE}} + V \cdot c_{\mathrm{ASIC}} = V \cdot c_{\mathrm{GPU}} \quad \Rightarrow \quad V^* = \frac{C_{\mathrm{NRE}}}{c_{\mathrm{GPU}} - c_{\mathrm{ASIC}}} \tag{2}$$

Cloud TPUs and smartphone NPUs justify specialization through **aggregate fleet volume** and **power envelopes** rather than per-chip sales alone. FPGAs occupy a middle ground: reprogrammable without full ASIC NRE, but typically lower clock frequency and higher power than hardened logic for the same function.

C. When SIMT Leaves Performance on the Table

GPUs pay overhead for generality: instruction fetch and decode per warp, register files sized for arbitrary thread counts, branch divergence handling, and kernel launch latency. When a workload satisfies all of the following, a DSA typically wins:

- **Stable algorithm** — the computation graph changes infrequently (months or years, not per request).
- **Regular dataflow** — predictable, dense memory access (systolic streaming, line buffers) rather than pointer chasing.
- **High arithmetic intensity** on a narrow operator set (GEMM, MAC arrays, DCT butterflies).
- **Sustained batch or stream** — sufficient work to fill the accelerator pipeline continuously.
- **Power or latency budget** — mobile milliwatts, datacenter TCO, or real-time line-rate constraints.

If any condition fails—small batches, frequent graph changes, irregular sparsity, or one-off prototypes—a GPU (or CPU) remains the rational choice.

—

III. Tensor Processing Units and Systolic Arrays

A. Motivation: ML as Dense Linear Algebra

Neural network training and inference reduce overwhelmingly to **generalized matrix multiplication** (GEMM) and convolutions (im2col + GEMM). A fully connected layer computes $Y = XW + b$; a convolution expands to a batched GEMM after im2col. Backpropagation is likewise dominated by GEMM and its transposed variants. GPUs accelerate these via SIMT thread blocks and tensor cores; TPUs hardwire the inner multiply–accumulate loop into a **systolic array**.

B. Systolic Array Execution

A systolic array is a mesh of processing elements (PEs) where data flows rhythmically through nearest neighbors with minimal off-chip memory traffic. For $C = A \times B$ with $A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{K \times N}$, rows of A and columns of B are streamed through an $R \times R$ array of PEs. Each PE p_{ij} holds a partial sum and performs:

$$c_{ij} \leftarrow c_{ij} + a_i \cdot b_j \tag{3}$$

as a_i propagates horizontally and b_j vertically. After K clock cycles (for an $R \times R$ tile with appropriate skew), a tile of C is accumulated. The key property is **reuse in place**: each loaded element participates in R MACs before leaving the array, maximizing arithmetic intensity without a large register file per thread.

Peak throughput on an array of size R at frequency f is approximately:

$$P_{\text{systolic}} \approx 2R^2 f \quad \text{(FLOP/s per array, multiply–add counted as 2 FLOP)} \tag{4}$$

Google's TPU v4i uses multiple large systolic arrays with High-Bandwidth Memory (HBM); TPU pods interconnect chips via toroidal networks for scaled training.

C. TPU vs. GPU: Structural Differences

- **Unified compiler stack** — XLA compiles TensorFlow/JAX graphs directly to TPU instructions; the programmer rarely writes kernels.
- **Deterministic dataflow** — no SIMT divergence, no occupancy tuning; batch dimensions must align with array geometry.
- **On-chip activation buffering** — scratchpad memory holds tiles of activations and weights; off-chip bandwidth is scheduled statically.
- **bfloat16 and int8 native paths** — reduced precision is first-class, not an optional tensor-core mode.

For large-batch training of dense transformers, TPU pods often achieve superior **performance per dollar and per watt** because the entire chip is devoted to sustained GEMM. For irregular workloads—dynamic shapes, sparse attention patterns, reinforcement-learning environments with CPU-bound simulation—GPUs retain flexibility advantages.

D. Roofline Comparison

Applying the roofline model, a TPU's ridge point $\text{AI}_{\text{ridge}} = P_{\text{peak}}/B_{\text{mem}}$ is tuned for GEMM-heavy kernels. Operations below the ridge (elementwise activations, layer normalization with low reuse) may underutilize the systolic array unless **fused** into adjacent GEMM tiles by the compiler. GPUs handle unfused low-AI kernels more gracefully via high occupancy across concurrent warps, but at higher energy cost.

—

IV. Neural Processing Units and Edge Inference

A. The Edge Constraint

Smartphone, wearable, and embedded SoCs cannot attach datacenter GPUs. Inference must run within a **thermal design power (TDP)** of 1–5 W for sustained use, with peak bursts for camera and voice workloads. NPUs (Apple Neural Engine, Qualcomm Hexagon, Arm Ethos, Samsung NPU) are DSAs integrated alongside CPU and GPU on the same die, optimized for **low-batch, low-precision inference**.

B. Typical NPU Architecture

NPUs combine:

- **MAC arrays** — smaller than TPU systolic arrays (hundreds to low thousands of MACs), clocked for energy efficiency.
- **Direct memory access (DMA) engines** — prefetch weights and activations into local SRAM while compute proceeds.
- **Fixed-function units** — depthwise convolutions, pooling, activation lookup tables, and Winograd transforms.
- **Weight compression** — on-the-fly decompression of int4/int8 quantized weights to reduce DRAM bandwidth.

Effective inference latency for a layer with N_{MAC} multiply–accumulates and peak NPU throughput P_{NPU} is bounded by:

$$T_{\text{layer}} \geq \max\left(\frac{2N_{\text{MAC}}}{P_{\text{NPU}}},\ \frac{D_{\text{weights}} + D_{\text{acts}}}{B_{\text{mem}}}\right) \tag{5}$$

where D_{weights} and D_{acts} are bytes moved and B_{mem} is local or DRAM bandwidth. NPUs win when the model fits supported operators and quantization schemes; unsupported ops **fall back** to CPU or GPU, fragmenting the pipeline.

C. NPU vs. GPU on Mobile

Mobile GPUs (Adreno, Mali, Apple GPU) are SIMT-capable and run general compute shaders, but draw higher power per inference token than a matched NPU. NPUs excel at:

- Camera pipeline models (face detection, segmentation, denoising) with fixed graphs.
- Always-on voice triggers and keyword spotting (tiny models, strict latency).
- Int8/int4 quantized vision and small-language-model inference.

GPUs remain preferable for gaming graphics, custom shader research, and models requiring operators absent from the NPU compiler (exotic activations, dynamic control flow, training).

—

V. FPGAs and Reconfigurable Acceleration

A. Architecture and Programming Model

A Field-Programmable Gate Array implements computation as a configurable network of lookup tables (LUTs), flip-flops, DSP slices (hard multipliers), and block RAM. Unlike a GPU kernel that executes sequentially over threads, an FPGA **pipeline** processes one sample or pixel per clock cycle once the pipeline is full—achieving deterministic, stream-processing latency.

Throughput for a fully pipelined design processing one element per cycle at frequency f is:

$$P_{\text{FPGA}} = f \cdot \text{ops per element} \tag{6}$$

Latency to first output is the pipeline depth L (cycles), but sustained throughput is f regardless of L once filled.

B. High-Level Synthesis and Dataflow

Modern FPGA development uses High-Level Synthesis (HLS) from C/C++ or OpenCL, or frameworks such as Vitis AI. The designer expresses loop nests with `#pragma HLS pipeline` and `#pragma HLS dataflow`, mapping arrays to BRAM and loops to spatial pipelines. This suits:

- **Streaming DSP** — FIR filters, FFT stages, custom modems.
- **Network packet processing** — fixed header parsing at line rate.
- **Prototype ASICs** — validate an algorithm in silicon before committing to NRE.
- **Low-volume specialized codecs** — custom entropy coders or niche video formats.

C. FPGA vs. GPU and ASIC

- **vs. GPU** — FPGAs win on deterministic latency, wire-speed streaming I/O, and bit-exact custom arithmetic at moderate parallelism; GPUs win on peak FLOP/s for large batched linear algebra and software ecosystem maturity.
- **vs. ASIC** — FPGAs avoid NRE respin but consume more area and power per function; at high volume, hardened ASICs dominate TCO.

Microsoft's Project Catapult and Amazon F1 instances exemplify datacenter FPGA deployment for search ranking and custom networking; these workloads have stable logic, moderate parallelism, and value **throughput per watt** over raw programmability.

—

VI. Fixed-Function Media ASICs and HEVC

A. The Encode–Decode Asymmetry

As established in the HEVC analysis, video encoding is a combinatorial rate–distortion search over partition modes, motion vectors, transform coefficients, and quantization parameters. Decoding executes only the inverse path specified in the bitstream. Encoder complexity exceeds decoder complexity by one to two orders of magnitude. Real-time encoding at 4K/60 therefore demands hardware assistance; software encoders on CPU or GPU remain viable for offline transcoding with unbounded search depth.

B. Hardware Encoder Pipeline

A fixed-function HEVC (or H.264/AV1) encoder ASIC implements hardened blocks for:

- **Motion estimation** — hierarchical integer and sub-pel SAD/SSD search with early termination heuristics.
- **Transform and quantization** — separable DCT/DST butterfly networks and QP-dependent quantizers.
- **Entropy coding** — CABAC or equivalent bitstream packing (encode-side only).
- **In-loop filtering** — deblocking and SAO applied before reference picture insertion.

Each block is a dedicated datapath operating on line buffers or CTU-sized windows, not a SIMT kernel launching thousands of threads per CTU. NVIDIA NVENC, Intel Quick Sync Video (QSV), Apple Media Engine, and AMD VCE embed such blocks alongside programmable GPU compute.

C. Why SIMT Struggles with Encode

Motion search over a ± 128 pixel window with quarter-pel interpolation for a 64×64 CTU involves irregular early-exit branches, variable partition decisions, and frequent reference memory access with poor cross-thread reuse. Mapping this to SIMT yields:

$$\text{Effective GPU encode throughput} \ll P_{\text{peak,GPU}} \cdot \eta_{\text{divergence}} \cdot \eta_{\text{occupancy}} \tag{7}$$

where $\eta_{\text{divergence}}, \eta_{\text{occupancy}} < 1$ penalize branchy search logic. A hardened ME engine achieves near-constant cycles per CTU independent of warp scheduling.

Decode, by contrast, is lighter and increasingly handled by fixed-function units on integrated graphics; the residual add, inverse transform, and motion compensation map more cleanly to SIMD or SIMT, though dedicated decode ASICs still dominate power efficiency for mobile playback.

D. GPU Role in Video Pipelines

GPUs remain essential for:

- **Pre/post processing** — scaling, color conversion, denoising, AI-based enhancement (super-resolution, artifact reduction).
- **Custom or research encoders** — when standard ASICs lack required rate-control or quality features.
- **Decode + GPU compute fusion** — rendering decoded frames as textures without CPU copies.

The CPU–GPU–ASIC boundary in video is defined by **which pipeline stages are algorithmically frozen in silicon** versus which require ongoing software evolution.

—

VII. Decision Framework: Specialization vs. SIMT

A. Workload Checklist

Table I summarizes accelerator fit. Use it as a heuristic, not a rigid taxonomy.

TABLE I
ACCELERATOR FIT BY WORKLOAD CHARACTERISTIC

Characteristic	Prefer DSA/ASIC	Prefer GPU SIMT
Algorithm stability	Fixed for product lifetime	Evolving weekly
Batch / stream size	Large, sustained	Small, bursty
Control flow	Regular, predictable	Branchy, divergent
Memory pattern	Streaming, tiled	Irregular, sparse
Precision	int8/bf16 native	Mixed, experimental
Volume / NRE	High fleet or mobile TDP	Low, prototype
Latency target	Fixed line rate (e.g., 16 ms)	Soft, throughput-oriented

B. Energy–Delay Product

For deployment at scale, total cost of ownership favors the architecture minimizing:

$$\mathrm{EDP} = E_{\mathrm{op}} \cdot T_{\mathrm{latency}} \tag{8}$$

DSAs reduce E_{op} on-target; GPUs reduce T_{latency} for diverse parallel kernels via massive parallelism. Hybrid systems assign each sub-problem to the minimizing architecture and accept **partitioning overhead** at interfaces (PCIe, shared memory, graph partitioning).

C. The Heterogeneous System Equation

Extending the CPU–GPU hybrid model, end-to-end latency on a system with CPU, GPU, TPU/NPU, and media ASIC is:

$$T_{\mathrm{total}} = T_{\mathrm{CPU,control}} + \sum_{k} \left(T_{\mathrm{transfer},k} + T_{\mathrm{accel},k} + T_{\mathrm{sync},k} \right) \tag{9}$$

Specialization wins only when $T_{\mathrm{accel},k}$ (and associated energy) savings exceed transfer and synchronization costs. Keeping tensors resident on-device across training steps, fusing operators in compiler graphs, and colocating media ASICs with GPU memory (zero-copy decode surfaces) are standard techniques to satisfy this inequality.

—

VIII. Case Studies

A. Large-Language-Model Inference

Transformer inference is memory-bandwidth-bound during autoregressive decoding (batch size 1, sequential token generation) and compute-bound during prefill (processing the prompt in parallel). Datacenter deployments use:

- **GPU tensor cores** — flexible, strong ecosystem (vLLM, TensorRT-LLM), excellent for mixed batch sizes.
- **TPU/GPU-class ASICs (Google TPU, AWS Trainium/Inferentia)** — optimized for sustained batch inference and int8/bf16 GEMM at fleet scale.
- **Mobile NPUs** — quantized small models (1–7B parameters, int4) for on-device assistants.

Speculative decoding and mixture-of-experts routing reintroduce irregularity, temporarily favoring programmable GPUs until specialized routing hardware matures.

B. Video Transcoding at Scale

A cloud transcoding farm receiving user uploads typically pipelines: demux (CPU) $\rightarrow$ decode (ASIC or GPU fixed-function) $\rightarrow$ optional GPU filter $\rightarrow$ encode (NVENC/QSV ASIC for speed, x265 CPU/GPU for quality tier). The **quality tier** selects programmable versus fixed-function encode: ASICs for live streaming latency; software encoders for archival quality with custom rate–distortion tuning.

C. HEVC and the Silicon Stack

A smartphone recording 4K video routes camera frames through an Image Signal Processor (ISP), then to a hardware HEVC encoder ASIC, bypassing the GPU entirely for the compression hot path. Playback inverts the chain via hardware decode. The GPU handles UI composition and effects. This three-way partition—CPU for orchestration, ASIC for codec line rate, GPU for graphics—is the canonical heterogeneous media stack.

—

IX. Conclusion

Domain-specific accelerators are not universal speedups; they are **contractual bargains** with silicon. The designer commits to a stable algorithm, regular dataflow, and sufficient volume (or a strict power envelope); the hardware returns lower energy per operation and higher sustained throughput than SIMT generality can provide.

TPUs and systolic arrays dominate large-scale dense linear algebra when compiler-managed dataflow aligns with array geometry. NPUs deliver milliwatt-class inference on mobile SoCs for supported quantized graphs. FPGAs bridge prototype and production for streaming, line-rate custom logic. Fixed-function media ASICs implement HEVC and successor codecs at real-time line rates where branchy motion search defeats GPU occupancy.

The decision criterion parallels CPU versus GPU analysis: match the processor class to **algorithm stability**, **arithmetic structure**, **memory regularity**, and **deployment economics**. Modern systems combine all classes—CPU control plane, GPU general parallel compute, TPU/NPU structured inference, FPGA reconfigurable I/O, and media ASIC fixed pipelines—into a single heterogeneous stack. Understanding when specialization beats SIMT is essential for principled design in machine learning infrastructure, mobile AI, cloud transcoding, and real-time signal processing.

References

[1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.

[2] N. P. Jouppi *et al.*, "In-Datacenter Performance Analysis of a Tensor Processing Unit," in *Proc. ISCA*, 2017, pp. 1–12.

[3] H. T. Kung, "Why Systolic Architectures?" *IEEE Computer*, vol. 15, no. 1, pp. 37–46, Jan. 1982.

[4] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[5] G. J. Sullivan *et al.*, "Overview of the High Efficiency Video Coding (HEVC) Standard," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 22, no. 12, pp. 1649–1668, Dec. 2012.

[6] AMD/Xilinx, "UltraScale Architecture and Product Overview," Product Guide, 2020.